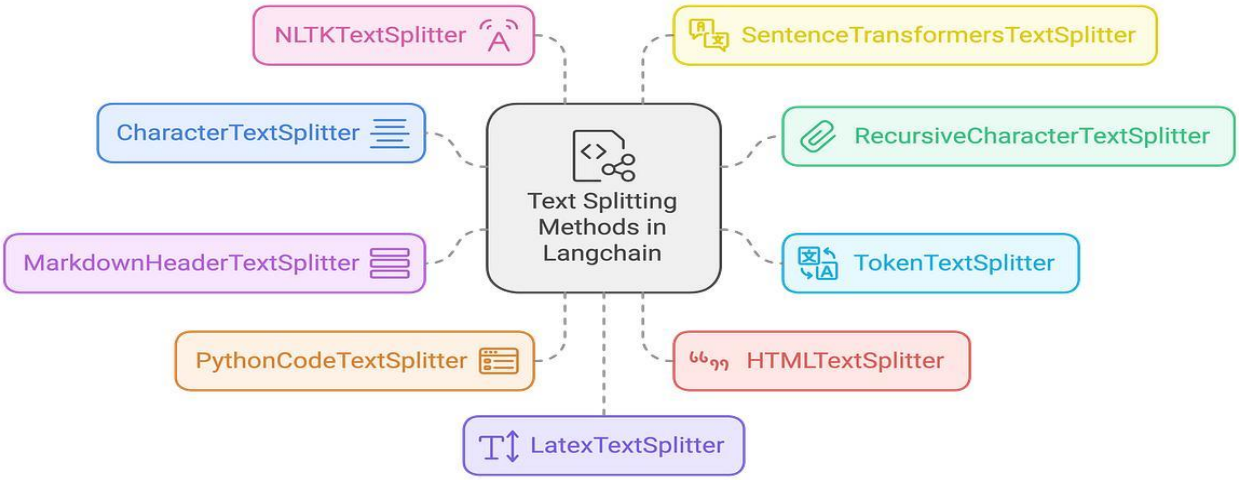
Text Splitting & Vector Store in RAG (LangChain)

🔹 What is Text Splitting?

Text Splitting is the process of breaking large chunks of text (like PDFs, logs, HTML pages, or books) into smaller, manageable pieces (**chunks**) that an LLM can process effectively.



Why Text Splitting is Needed ??

1. Overcoming Model Limitations:

- LLMs and embedding models have token limits
- Large documents cannot be processed directly

--> Splitting ensures data fits within model constraints

2. Improves Downstream Tasks:

Task	Why Splitting Helps
Embedding	Smaller chunks → better vector representation
Semantic Search	More precise retrieval
Summarization	Prevents topic mixing

3. Optimizing Performance:

- Smaller chunks → faster processing
- Enables parallelization
- Reduces memory usage

🔹 Types of Text Splitters:

1. Length-Based Text Splitting

Splits text based on fixed size (characters/tokens).

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(text)
```

Limitations:

- Breaks sentences in between
- Loses semantic meaning
- Context fragmentation

2. Text Structure-Based Splitting

Splits based on natural structure.

- Paragraphs
- Sentences
- Line breaks

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(text)
```

Limitations:

- Structure may not always exist (e.g., logs)
- Still may break context if chunk size exceeded
- Not fully semantic

3. Document Structure-Based Splitting

Splits based on document hierarchy

- Headings

- Sections
- Code blocks

```
from langchain.text_splitter import MarkdownHeaderTextSplitter

headers = [("#", "Header1"), ("##", "Header2")]

splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers)

chunks = splitter.split_text(markdown_text)
```

Limitations:

- Depends on well-structured documents
- Not suitable for unstructured text
- May create uneven chunk sizes

4. Semantic Meaning-Based Splitting

Splits text based on meaning/context similarity
Uses embeddings to group related sentences

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Often combined with embeddings externally
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_text(text)
```

Limitations:

- Computationally expensive
- Requires embedding models
- Slower than other methods